

计算几何

Morning Flower and Evening Oath

XCPC vector 模板分册

目录

使用约定	iii
第一章 二维计算几何基础	1
1.1 浮点点向量、圆与交点结果	1
1.2 点在直线上的投影	2
1.3 浮点两直线交点	3
1.4 浮点直线与圆交点	4
1.5 浮点两圆交点	5
1.6 两圆交面积	6
1.7 整数点向量与线段判定	6
1.8 整数多边形有向面积	7
1.9 点在整数简单多边形内	8
1.10 圆与简单多边形相交面积	9
1.11 两圆公切线与过点切线	11
第二章 三维计算几何基础	13
2.1 整数三维共线、共面与方向	13
第三章 距离	14
3.1 点到闭线段的距离	14
第四章 凸包	15
4.1 整数凸包：单调链	15
4.2 点在整数凸包内（二分）	16
4.3 凸多边形 Minkowski 和	17
4.4 支撑点查询恢复下凸包	19
第五章 旋转卡壳	21
5.1 整数凸包直径：旋转卡壳	21
第六章 半平面交	22
6.1 整数半平面的有界多边形交	22
第七章 平面最近点对	25
7.1 整数最近点对（精确平方距离）	25
7.2 浮点平面最近点对	26
第八章 随机增量法	28
8.1 随机增量最小覆盖圆	28
第九章 配套知识与验证范围	30
9.1 数值算法与几何	30

第十章 计算几何：精确判定与适用范围	31
10.1 整数系数的有界半平面交	31
10.2 最近点对、Minkowski 和与三维整数判定	32
10.3 浮点最近点对	33
10.4 公切线与过点切线	34
10.5 随机增量最小覆盖圆	35
10.6 圆与简单多边形相交面积	36

使用约定

C++20，默认包含 `bits/stdc++.h` 与 `cassert`，并使用 `std` 命名空间。下标、区间及数值范围以各条目说明为准。DFS 保持递归，需满足题目栈空间条件。

本册主目录沿用 OI Wiki 官方导航的大、中、小层级，各页下列本库实现。PBDS 与 GNU rope 按队内约定纳入平衡树。组合条目与相关挂靠仍见仓库分类审计。跨类引用的已收录组件附在依赖部分；源头文件中的依赖仍须一并检查。

本册尚非全量完成稿；传统版评测仅为历史档案，当前只维护 `vector` 代码。

第一章 二维计算几何基础

1.1 浮点点向量、圆与交点结果

R 为 long double; Point 支持向量加减、数乘和数除 (除数非零)。dot/cross/norm/perp 为基本运算, norm 使用 hypotl; Circle 保存圆心和半径。Kind 为 none、one、two、infinite、degenerate; Result 同时保存类别与有限交点。输入须有限, 尺度须使后续平方、乘积不溢出或下溢; 排序不使用 epsilon。具体容差与退化约定见各函数。

```
1 struct RealPlane
2 {
3     using R = long double;
4
5     struct Point
6     {
7         R x = 0, y = 0;
8
9         Point operator+(Point b) const { return {x + b.x, y + b.y}; }
10
11        Point operator-(Point b) const { return {x - b.x, y - b.y}; }
12
13        Point operator*(R k) const { return {x * k, y * k}; }
14
15        Point operator/(R k) const { return {x / k, y / k}; }
16    };
17
18    struct Circle
19    {
20        Point o;
21        R r;
22    };
```

```

23     enum class Kind
24     {
25         none,
26         one,
27         two,
28         infinite,
29         degenerate
30     };
31
32     struct Result
33     {
34         Kind kind;
35         vector<Point> p;
36     };
37
38     static R dot(Point a, Point b) { return a.x * b.x + a.y * b.y; }
39
40     static R cross(Point a, Point b) { return a.x * b.y - a.y * b.x; }
41
42     static R norm(Point a) { return hypot1(a.x, a.y); }
43
44     static Point perp(Point a) { return {-a.y, a.x}; }
45 };

```

1.2 点在直线上的投影

`line_projection(p,a,b)` 返回 `p` 在过 `a`、`b` 直线上的正交投影；若 `a=b`，则约定返回 `a`。类型为 `RealPlane::Point`。输入有限且尺度适当，平方和与点积须能可靠表示；不使用 `epsilon` 把短线段自动视为零。 $O(1)$ 时间和空间。

```

1 RealPlane::Point
2 line_projection(RealPlane::Point p, RealPlane::Point a, RealPlane::Point b)
3 {
4     using G = RealPlane;
5     using R = G::R;
6     using Point = G::Point;
7     Point v = b - a;
8     R d = G::dot(v, v);
9     if (d == 0) return a;
10    return a + v * (G::dot(p - a, v) / d);
11 }

```

1.3 浮点两直线交点

`line_intersection_real(a,b,c,d,eps=1e-12L)` 返回 `RealPlane::Result`; `eps` 必须为正。端点重合使任一直线退化时返回 `degenerate`。按方向叉积与方向长度乘积的相对阈值判断平行，再按位置阈值区分 `none` 与 `infinite`；唯一交点返回 `one`。输入有限且尺度适当；近似分类不等同于精确共线判定。O(1)。

```

1 RealPlane::Result line_intersection_real(RealPlane::Point a,
2                                         RealPlane::Point b,
3                                         RealPlane::Point c,
4                                         RealPlane::Point d,
5                                         long double eps = 1e-12L)
6 {
7     using G = RealPlane;
8     using R = G::R;
9     using Point = G::Point;
10    using Kind = G::Kind;
11    assert(eps > 0);
12    Point u = b - a, v = d - c;
13    R nu = G::norm(u), nv = G::norm(v);
14    if (nu == 0 || nv == 0) return {Kind::degenerate, {}};
15    R det = G::cross(u, v);
16    if (fabsl(det) <= eps * nu * nv)
17        return {fabsl(G::cross(c - a, u)) <= eps * nu * max(R(1), G::norm(c - a))
18                ? Kind::infinite
19                : Kind::none,
20                {}};
21    return {Kind::one, {a + u * (G::cross(c - a, v) / det)}};
22 }

```

1.4 浮点直线与圆交点

`line_circle_intersections(a,b,c,eps=1e-12L)` 返回 `Result`; `c` 为 `RealPlane::Circle`, 半径非负, `eps` 为正。`a=b` 返回 `degenerate`; 按投影点到圆心的距离和半径的相对阈值区分 `none`、`one`、`two`。点圆允许; 只有正半径同心圆才涉及无限交点, 本函数不会返回 `infinite`。依赖 `line_projection`, $O(1)$ 。输入有限且尺度适当, 接近相切时按容差归类。

```

1 RealPlane::Result line_circle_intersections(RealPlane::Point a,
2                                             RealPlane::Point b,
3                                             RealPlane::Circle c,
4                                             long double eps = 1e-12L)
5 {
6     using G = RealPlane;
7     using R = G::R;
8     using Point = G::Point;
9     using Kind = G::Kind;
10    assert(eps > 0);
11    assert(c.r >= 0);
12    Point v = b - a;
13    R len = G::norm(v);
14    if (len == 0) return {Kind::degenerate, {}};
15    Point h = line_projection(c.o, a, b);
16    R d = G::norm(h - c.o), tol = eps * max(d, c.r);
17    if (d > c.r + tol) return {Kind::none, {}};
18    if (fabs1(d - c.r) <= tol) return {Kind::one, {h}};
19    R t = sqrtl(max(R(0), (c.r - d) * (c.r + d)));
20    Point w = v * (t / len);
21    return {Kind::two, {h - w, h + w}};
22 }

```

1.5 浮点两圆交点

`circle_intersections(a,b,eps=1e-12L)` 返回 `Result`; 半径非负, `eps` 为正。两个完全相同的正半径圆返回 `infinite`; 同位置两个点圆返回 `one`; 同心不同半径返回 `none`。其余用距离和半径的相对阈值分类, 点圆单独处理, 接近相切可返回 `one`。输入有限且尺度适当, $O(1)$ 时间空间; 返回的有限交点仍受浮点构造误差影响。

```

1 RealPlane::Result
2 circle_intersections(RealPlane::Circle a, RealPlane::Circle b, long double eps = 1e-12L)
3 {
4     using G = RealPlane;
5     using R = G::R;
6     using Point = G::Point;
7     using Kind = G::Kind;
8     assert(eps > 0);
9     assert(a.r >= 0 && b.r >= 0);
10    Point v = b.o - a.o;
11    R d = G::norm(v);
12    R tol = eps * max({d, a.r, b.r});
13    // Infinite intersections require identical positive-radius circles.
14    if (d == 0)
15    {
16        if (a.r != b.r) return {Kind::none, {}};
17        if (a.r == 0) return {Kind::one, {a.o}};
18        return {Kind::infinite, {}};
19    }
20    if (a.r == 0 && b.r == 0) return {Kind::none, {}};
21    if (a.r == 0 || b.r == 0)
22    {
23        R radius = max(a.r, b.r);
24        if (fabsl(d - radius) > tol) return {Kind::none, {}};
25        return {Kind::one, {a.r == 0 ? a.o : b.o}};
26    }
27    if (d > a.r + b.r + tol || d < fabsl(a.r - b.r) - tol) return {Kind::none, {}};
28    R x = (d * d + (a.r - b.r) * (a.r + b.r)) / (2 * d);
29    Point h = a.o + v * (x / d);
30    // Near-concentric circles must not be mistaken for internal tangency.
31    if (fabsl(x) > a.r + tol) return {Kind::none, {}};
32    if (fabsl(fabsl(x) - a.r) <= tol) return {Kind::one, {h}};
33    R y = sqrtl(max(R(0), (a.r - x) * (a.r + x)));
34    Point w = G::perp(v) * (y / d);
35    return {Kind::two, {h - w, h + w}};
36 }

```

1.6 两圆交面积

`circle_overlap_area(a,b)` 返回 long double 面积，半径非负。分离或外切返回 0；内含时返回小圆面积，含点圆和同心圆。部分相交计算两个圆弓形面积，小角度使用展开式减轻相消；`acos` 入参截断至 $[-1,1]$ 。不使用交点分类容差；要求有限且尺度适当的输入。O(1) 时间空间，不把浮点输出视为精确面积。

```

1 RealPlane::R circle_overlap_area(RealPlane::Circle a, RealPlane::Circle b)
2 {
3     using G = RealPlane;
4     using R = G::R;
5     assert(a.r >= 0 && b.r >= 0);
6     R d = G::norm(a.o - b.o), pi = acosl(-1.L);
7     if (d >= a.r + b.r) return 0;
8     if (d <= fabs1(a.r - b.r)) return pi * min(a.r, b.r) * min(a.r, b.r);
9     R difference = (a.r - b.r) * (a.r + b.r);
10    R x = acosl(clamp((d * d + difference) / (2 * d * a.r), R(-1), R(1)));
11    R y = acosl(clamp((d * d - difference) / (2 * d * b.r), R(-1), R(1)));
12    auto segment = [](R angle)
13    {
14        if (angle < 1e-3L)
15        {
16            R z = angle * angle;
17            return angle * z * (R(2) / 3 + z * (-R(2) / 15 + z * R(4) / 315));
18        }
19        return angle - sinl(2 * angle) / 2;
20    };
21    return a.r * a.r * segment(x) + b.r * b.r * segment(y);
22 }
```

1.7 整数点向量与线段判定

坐标绝对值不超过 10^{12} ，使用 long long 存点、`__int128_t` 计算叉积、点积与距离平方。Point 按 x、y 字典序比较；`cross(a,b,c)` 为 ab 与 ac 的叉积，`dot` 为点积。`on_segment` 包含端点及退化线段；`intersect` 判断闭线段有交点。PolarLess 按半平面、叉积、长度平方排序，要求严格弱序。全部单次运算 O(1)，不使用 epsilon。

```

1 struct IntegerPlane
2 {
3     using ll = long long;
4     using I = __int128_t;
5
6     // Contract: |coordinate|<=1e12. Differences/products widened before arithmetic.
7     struct Point
8     {
9         ll x = 0, y = 0;
10
11         bool operator<(Point b) const { return tie(x, y) < tie(b.x, b.y); }
12
13         bool operator==(Point b) const { return x == b.x && y == b.y; }
14     };
15
16     static I cross(Point a, Point b, Point c)
17     {
18         return (I(b.x) - a.x) * (I(c.y) - a.y) - (I(b.y) - a.y) * (I(c.x) - a.x);
19     }
20 }
```

```

21 static I dot(Point a, Point b, Point c)
22 {
23     return (I(b.x) - a.x) * (I(c.x) - a.x) + (I(b.y) - a.y) * (I(c.y) - a.y);
24 }
25
26 static I dist2(Point a, Point b) { return dot(a, b, b); }
27
28 static int sign(I x) { return (x > 0) - (x < 0); }
29
30 static bool on_segment(Point p, Point a, Point b)
31 {
32     return cross(a, b, p) == 0 && dot(p, a, b) <= 0;
33 }
34
35 static bool intersect(Point a, Point b, Point c, Point d)
36 {
37     int x = sign(cross(a, b, c)), y = sign(cross(a, b, d));
38     int z = sign(cross(c, d, a)), w = sign(cross(c, d, b));
39     if (!x && on_segment(c, a, b)) return true;
40     if (!y && on_segment(d, a, b)) return true;
41     if (!z && on_segment(a, c, d)) return true;
42     if (!w && on_segment(b, c, d)) return true;
43     return x * y < 0 && z * w < 0;
44 }
45
46 struct PolarLess
47 {
48     static int half(Point p) { return p.y < 0 || (p.y == 0 && p.x < 0); }
49
50     bool operator()(Point a, Point b) const
51     {
52         if (half(a) != half(b)) return half(a) < half(b);
53         I c = cross({0, 0}, a, b);
54         if (c) return c > 0;
55         return dist2({0, 0}, a) < dist2({0, 0}, b);
56     }
57 };
58 };

```

1.8 整数多边形有向面积

`polygon_area2(p)` 返回 `__int128_t` 的有向面积两倍；逆时针为正，反向为负。输入按边界顺序排列，隐式闭合；空集或不足三个点返回 0。单个多边形点数不超过 10^5 、坐标绝对值不超过 10^{12} ，确保求和在有符号 128 位内。时间 $O(n)$ ，额外空间 $O(1)$ 。

```

1 IntegerPlane::I polygon_area2(const vector<IntegerPlane::Point> &p)

```

```

2 {
3     using G = IntegerPlane;
4     using I = G::I;
5     I ans = 0;
6     int n = (int)p.size();
7     for (int i = 0; i < n; i++)
8         ans += I(p[i].x) * p[(i + 1) % n].y - I(p[i].y) * p[(i + 1) % n].x;
9     return ans;
10 }

```

1.9 点在整数简单多边形内

`polygon_contains(p,q)` 返回 0 外部、1 边界、2 内部，输入简单多边形，可顺时针或逆时针，不重复首点。空多边形返回 0；单点与线段按闭集判断边界。绕数算法先判线段，再按半开纵坐标区间累计，避免顶点重复计数。坐标范围同 `IntegerPlane`， $O(n)$ 时间、 $O(1)$ 空间。

```

1 int polygon_contains(const vector<IntegerPlane::Point> &p, IntegerPlane::Point q)
2 {
3     using G = IntegerPlane;
4     using Point = G::Point;
5     using I = G::I;
6     int winding = 0, n = (int)p.size();
7     for (int i = 0; i < n; i++)
8     {
9         Point a = p[i], b = p[(i + 1) % n];
10        if (G::on_segment(q, a, b)) return 1;
11        I c = G::cross(a, b, q);
12        if (a.y <= q.y && b.y > q.y && c > 0) ++winding;
13        if (a.y > q.y && b.y <= q.y && c < 0) --winding;
14    }
15    return winding ? 2 : 0;
16 }

```

1.10 圆与简单多边形相交面积

`area(points, circle)` 返回非负相交面积；多边形须简单，允许凹多边形、顺时针/逆时针和相邻重复顶点。少于三点或半径零返回零。`edge(a, b, r)` 返回圆心在 origin 时有向边 `a` 到 `b` 的面积贡献，`r` 必须非负。按交点分段，内部用三角形，外部用有向扇形；补偿求和， $O(n)$ 时间、 $O(1)$ 额外空间。依赖浮点几何的 `Point/Circle` 和向量运算。输入须有限、尺度适当，平移到圆心后作差、乘积和面积均须可表示；极小交叠及接近相切时仍有浮点误差。

```

1 struct CirclePolygon
2 {
3     using G = RealPlane;
4     using R = G::R;
5     using Point = G::Point;
6     using Circle = G::Circle;
7
8     // Signed contribution of edge a -> b; circle centered at the origin.
9     static R edge(Point a, Point b, R r)
10    {
11        if (r == 0) return 0;
12        Point v = b - a;
13        R length = G::norm(v);
14        if (length == 0) return 0;
15        Point u = v / length;
16        R h = G::cross(u, a);
17        Point cuts[4] = {a};
18        int count = 1;
19        if (fabs(h) <= r)
20        {
21            R delta = sqrtl(max(R(0), (r - fabs(h)) * (r + fabs(h))));
22            R t = -G::dot(a, u);
23            Point foot = G::perp(u) * h;
24            for (R s : {-delta, delta})
25                if (t + s > 0 && t + s < length) cuts[count++] = foot + u * s;
26        }
27        cuts[count++] = b;

```

分段面积与逐边累加（接上页同一结构体与函数）：

```

28     R answer = 0;
29     for (int i = 1; i < count; i++)
30     {
31         Point p = cuts[i - 1], q = cuts[i];
32         R cross = G::cross(p, q);
33         if (G::norm((p + q) / 2) <= r)
34             answer += cross / 2;
35         else
36             answer += r * r * atan2l(cross, G::dot(p, q)) / 2;
37     }
38     return answer;
39 }
40
41 // Simple polygon, either orientation; duplicate consecutive vertices allowed.
42 static R area(const vector<Point> &p, Circle c)
43 {
44     assert(c.r >= 0);
45     if (p.size() < 3 || c.r == 0) return 0;
46     R answer = 0, error = 0;
47     for (int i = 0; i < (int)p.size(); i++)
48     {
49         R value = edge(p[i] - c.o, p[(i + 1) % p.size()] - c.o, c.r);
50         R y = value - error;
51         R sum = answer + y;
52         error = (sum - answer) - y;
53         answer = sum;
54     }
55     return fabsl(answer);
56 }
57 };

```

1.11 两圆公切线与过点切线

`solve(a,b,eps)` 返回 `Result`，包含 `infinite` 和 `lines`；`infinite` 为真表示无穷多条，否则 `lines` 给出全部切线。每条 `Tangent` 保存切点 `a`、`b`，单位法向量 `normal`，以及 `side`。`a`、`b` 对应输入两圆，直线方程 $\text{dot}(\text{normal}, x - a) = 0$ ；切点相同时仍有效。`side=1` 外公切线、`-1` 内公切线、`0` 两个不同点。半径零表示点，`from_point(p,c,eps)` 等价于第一圆取点，圆上切点读 `b`。输入有限，坐标及半径绝对值 $\leq 1e100$ ，半径非负。`eps` 默认 `1e-12`，用于相对圆心距的近相切判断；近相切可能合并为一条，浮点坐标仍有舍入误差。严格同心等半径时 `infinite=true`，包括同一点；同心不等半径返回空。`O(1)`；依赖浮点几何的 `Point/Circle`、点运算及 `norm/perp`。

```

1 struct CircleTangents
2 {
3     using G = RealPlane;
4     using R = G::R;
5     using Point = G::Point;
6     using Circle = G::Circle;
7
8     struct Tangent
9     {
10         Point a, b, normal;
11         int side; // 1 outer, -1 inner, 0 two distinct point circles.
12     };
13
14     struct Result
15     {
16         bool infinite = false;
17         vector<Tangent> lines;
18     };
19
20     // Line: dot(normal, x - a) = 0. Radius zero represents a point.
21     static Result solve(Circle a, Circle b, R eps = 1e-12L)
22     {
23         assert(a.r >= 0 && b.r >= 0 && eps > 0 && eps < 1);
24         Point v = b.o - a.o;
25         R d = G::norm(v);
26         if (d == 0) return {a.r == b.r, {}};
27         Point u = v / d;
28         if (a.r == 0 && b.r == 0) return {false, {{a.o, b.o, G::perp(u), 0}}};

```

内外公切线枚举（接上页同一函数）：

```

29     Result answer;
30     for (int side : {1, -1})
31     {
32         if (side == -1 && (a.r == 0 || b.r == 0)) continue;
33         R gap = a.r - side * b.r;
34         if (fabs1(gap) > d * (1 + eps)) continue;
35         bool touch = fabs1(gap) >= d * (1 - eps);
36         R c = touch ? (gap < 0 ? -1 : 1) : gap / d;
37         R s = sqrt1(max(R(0), (1 - c) * (1 + c)));
38         for (int sign : {-1, 1})
39         {
40             Point normal = u * c + G::perp(u) * (sign * s);
41             Point p = a.o + normal * a.r;
42             Point q = b.o + normal * (side * b.r);
43             answer.lines.push_back({p, q, normal, side});
44             if (touch) break;
45         }
46     }
47     return answer;
48 }
49
50 static Result from_point(Point p, Circle c, R eps = 1e-12L)
51 {
52     return solve({p, 0}, c, eps);
53 }
54 };

```

第二章 三维计算几何基础

2.1 整数三维共线、共面与方向

坐标绝对值 $\leq 10^9$; orient(a,b,c,d) 为有向六倍四面体体积, 0 表示共面; collinear 判三点共线, on_segment 支持退化线段. int128 精确判定, 不包含三维凸包构造。

```
1 struct IntegerGeometry3D
2 {
3     using I = __int128_t;
4
5     // |coordinates|<=1e9: triple products fit signed int128.
6     struct Point
7     {
8         long long x, y, z;
9     };
10
11     struct Vector
12     {
13         I x, y, z;
14     };
15
16     static Vector diff(Point a, Point b)
17     {
18         return {I(a.x) - b.x, I(a.y) - b.y, I(a.z) - b.z};
19     }
20
21     static Vector cross(Vector a, Vector b)
22     {
23         return {a.y * b.z - a.z * b.y, a.z * b.x - a.x * b.z, a.x * b.y - a.y * b.x};
24     }
25
26     static I dot(Vector a, Vector b) { return a.x * b.x + a.y * b.y + a.z * b.z; }
27
28     static I orient(Point a, Point b, Point c, Point d)
29     {
30         return dot(cross(diff(b, a), diff(c, a)), diff(d, a));
31     }
32
33     static bool collinear(Point a, Point b, Point c)
34     {
35         auto v = cross(diff(b, a), diff(c, a));
36         return !v.x && !v.y && !v.z;
37     }
38
39     static bool on_segment(Point p, Point a, Point b)
40     {
41         return collinear(p, a, b) && dot(diff(a, p), diff(b, p)) <= 0;
42     }
43 };
```

第三章 距离

3.1 点到闭线段的距离

`segment_distance_real(p,a,b)` 返回点 p 到闭线段 ab 的 long double 距离。 $a=b$ 时返回到端点的距离；否则把投影参数截断至 $[0,1]$ 。要求有限且尺度适当的输入； $O(1)$ 时间和空间。

```
1 RealPlane::R
2 segment_distance_real(RealPlane::Point p, RealPlane::Point a, RealPlane::Point b)
3 {
4     using G = RealPlane;
5     using R = G::R;
6     using Point = G::Point;
7     Point v = b - a;
8     R d = G::dot(v, v);
9     if (d == 0) return G::norm(p - a);
10    R t = clamp(G::dot(p - a, v) / d, R(0), R(1));
11    return G::norm(p - (a + v * t));
12 }
```

第四章 凸包

4.1 整数凸包：单调链

`integer_hull(points)` 按值接收 `IntegerPlane::Point` 序列，输出严格逆时针凸包，不重复首点、不保留共线边内点。自动排序去重；全共线时保留两个端点，空集及单点直接返回。坐标范围同 `IntegerPlane`。 $O(n \log n)$ 时间、 $O(n)$ 空间；供凸包包含、直径与 Minkowski 和调用。

```
1 vector<IntegerPlane::Point> integer_hull(vector<IntegerPlane::Point> p)
2 {
3     using G = IntegerPlane;
4     using Point = G::Point;
5     sort(p.begin(), p.end());
6     p.erase(unique(p.begin(), p.end()), p.end());
7     int n = (int)p.size();
8     if (n < 3) return p;
9     vector<Point> h;
10    for (Point v : p)
11    {
12        while (h.size() > 1 && G::cross(h[h.size() - 2], h.back(), v) <= 0)
13            h.pop_back();
14        h.push_back(v);
15    }
16    int low = (int)h.size();
17    for (int i = n - 2; i >= 0; i--)
18    {
19        while ((int)h.size() > low && G::cross(h[h.size() - 2], h.back(), p[i]) <= 0)
20            h.pop_back();
21        h.push_back(p[i]);
22    }
23    h.pop_back();
24    return h; // CCW, no repeated endpoint, no collinear interior points.
25 }
```

4.2 点在整数凸包内（二分）

`convex_contains_i64(p,q)` 返回 0 外部、1 边界、2 内部。要求严格逆时针凸包，无共线边内点、不重复首点，例如 `integer_hull` 的输出；允许循环旋转起点。空集、单点、线段转用 `polygon_contains`。坐标范围同 `IntegerPlane`， $O(\log n)$ 时间、 $O(1)$ 空间。

```

1 int convex_contains_i64(const vector<IntegerPlane::Point> &p, IntegerPlane::Point q)
2 {
3     using G = IntegerPlane;
4     using I = G::I;
5     int n = (int)p.size();
6     if (n < 3) return polygon_contains(p, q);
7     I a = G::cross(p[0], p[1], q), b = G::cross(p[0], p[n - 1], q);
8     if (a < 0 || b > 0) return 0;
9     if (!a) return G::on_segment(q, p[0], p[1]) ? 1 : 0;
10    if (!b) return G::on_segment(q, p[0], p[n - 1]) ? 1 : 0;
11    int l = 1, r = n - 1;
12    while (l + 1 < r)
13    {
14        int m = (l + r) / 2;
15        if (G::cross(p[0], p[m], q) >= 0)
16            l = m;
17        else
18            r = m;
19    }
20    I c = G::cross(p[l], p[r], q);
21    return c < 0 ? 0 : c == 0 ? 1 : 2;
22 }
```

相关应用：固定导航没有独立 Minkowski 和条目，按凸包的相关应用归档。

4.3 凸多边形 Minkowski 和

`minkowski_sum(a,b)` 接受不重复首点的严格逆时针凸包；空输入返回空集，点和线段允许作为退化凸包。

点类型 `IntegerPlane::Point`；输入坐标及两集合坐标和的绝对值不超过 10^{12} 。结果不重复首点，非退化时沿逆时针边方向合并，平行同向边同时推进； $O(n+m)$ 时间。

退化输入枚举与另一凸包各点的和再求凸包， $O((n+m) \log(n+m))$ 。按值传入，内部旋转不修改调用方；依赖 `IntegerPlane` 的 `integer_hull/cross`。

```

1 vector<IntegerPlane::Point> minkowski_sum(vector<IntegerPlane::Point> a,
2                                           vector<IntegerPlane::Point> b)
3 {
4     using G = IntegerPlane;
5     using P = G::Point;
6     using I = G::I;
7     if (a.empty() || b.empty()) return {};
8     auto plus = [](P u, P v)
9     {
10         return P{u.x + v.x, u.y + v.y};
11     };
12     if (a.size() < 3 || b.size() < 3)
13     {
14         vector<P> p;
15         for (P u : a)
16             for (P v : b) p.push_back(plus(u, v));
17         return integer_hull(p);
18     }
19     auto start = [](vector<P> &p)
20     {
21         auto it = min_element(
22             p.begin(), p.end(), [](P u, P v) { return tie(u.y, u.x) < tie(v.y, v.x); });
23         rotate(p.begin(), it, p.end());
24     };
25     start(a);
26     start(b);
27     int n = a.size(), m = b.size(), i = 0, j = 0;
28     vector<P> result;
29     while (i < n || j < m)
30     {
31         result.push_back(plus(a[i % n], b[j % m]));
32         if (i == n)
33         {
34             ++j;
35             continue;
36         }
37         if (j == m)
38         {
39             ++i;
40             continue;

```

```
41     }
42     P u{a[(i + 1) % n].x - a[i].x, a[(i + 1) % n].y - a[i].y};
43     P v{b[(j + 1) % m].x - b[j].x, b[(j + 1) % m].y - b[j].y};
44     I c = G::cross({0, 0}, u, v);
45     if (c >= 0) ++i;
46     if (c <= 0) ++j;
47 }
48 return result;
49 }
```

4.4 支撑点查询恢复下凸包

静态 `build(left,right,oracle)` 返回从左到右的严格下凸包顶点，`Point` 为 $\{x,y\}$ 。有限非空整数点集无需显式给出；回调 `oracle(a,b)` 必须精确返回使 $a*x+b*y$ 最大的可行点，允许任意最优解。`left/right` 必须分别为最小/最大横坐标中纵坐标最低的点；全体同横坐标时二者必须相同。所有坐标绝对值不超过 10^{18} ，方向差使用 `long long`，叉积先提升至 `int128`；回调内部也须保证数值精确且无溢出。对弦 $a \rightarrow b$ 查询方向 $(b.y-a.y, a.x-b.x)$ ，严格位于弦下方则递归分治，否则确认边；输出时删除共线中间点。若分治找到 K 个严格改善支撑点，查询次数为 $2K+1$ （同横坐标时为 0），额外时间、空间和最坏递归深度为 $O(K+1)$ 。 K 包括可能返回的共线非顶点，不将其等同于最终顶点数。不能直接传浮点近似优化器。二分图闭包组合：点为 $(|S|, |N(S)|)$ ，左点赋权 a 、右点赋权 b ，左 $\rightarrow$ 邻点依赖；下凸包方向满足 $a \geq 0, b < 0$ ，最优闭包的右点恰为邻集。端点为 $(0,0)$ 和 $(\text{左点数}, \text{全部左点的邻点数})$ ，有孤立右点时后者不等于全部右点数。该组合已做子集枚举对拍，尚不代表关联赛题整题通过。

```

1 struct SupportHull
2 {
3     using ll = long long;
4     using I = __int128_t;
5
6     struct Point
7     {
8         ll x, y;
9     };
10
11     static I cross(Point a, Point b, Point c)
12     {
13         return (I(b.x) - a.x) * (I(c.y) - a.y) - (I(b.y) - a.y) * (I(c.x) - a.x);
14     }

```

```

16 // oracle(a,b) maximizes a*x+b*y exactly; |coordinates| <= 1e18.
17 // left/right: lowest points at the minimum/maximum x.
18 template <class Oracle>
19 static vector<Point> build(Point left, Point right, Oracle &&oracle)
20 {
21     assert(left.x <= right.x);
22     vector<Point> hull{left};
23     auto dfs = [&](auto &&self, Point a, Point b) -> void
24     {
25         Point c = oracle(b.y - a.y, a.x - b.x);
26         I turn = cross(a, b, c);
27         assert(turn <= 0);
28         if (turn < 0)
29         {
30             assert(a.x < c.x && c.x < b.x);
31             self(self, a, c);
32             self(self, c, b);
33         }
34         else
35         {
36             while (hull.size() >= 2 &&
37                 cross(hull[hull.size() - 2], hull.back(), b) == 0)
38                 hull.pop_back();
39             hull.push_back(b);
40         }
41     };
42     if (left.x < right.x)
43         dfs(dfs, left, right);
44     else
45         assert(left.y == right.y);
46     return hull;
47 }
48 };

```

第五章 旋转卡壳

5.1 整数凸包直径：旋转卡壳

`convex_diameter2(p)` 返回最远两点距离平方，类型 `__int128_t`；不足两点返回 0。输入须为严格逆时针凸包，不重复首点，例如 `integer_hull` 的输出。坐标范围同 `IntegerPlane`；旋转卡壳 $O(n)$ 时间、 $O(1)$ 额外空间，只返回距离，不返回端点编号。

```
1 IntegerPlane::I convex_diameter2(const vector<IntegerPlane::Point> &p)
2 {
3     using G = IntegerPlane;
4     using I = G::I;
5     int n = (int)p.size();
6     if (n < 2) return 0;
7     if (n == 2) return G::dist2(p[0], p[1]);
8     I ans = 0;
9     int j = 1;
10    for (int i = 0; i < n; i++)
11    {
12        int k = (i + 1) % n;
13        while (G::cross(p[i], p[k], p[(j + 1) % n]) > G::cross(p[i], p[k], p[j]))
14            j = (j + 1) % n;
15        ans = max({ans, G::dist2(p[i], p[j]), G::dist2(p[k], p[j])});
16    }
17    return ans;
18 }
```

第六章 半平面交

6.1 整数半平面的有界多边形交

`bounded_polygon(lines)` 求 $a*x+b*y \leq c$ 的交集闭包顶点, $\text{Line}=\{a,b,c\}$, 三个整数系数绝对值均 $\leq 10^9$ 。要求交集有界, 空集也允许; 只返回正面积多边形, 点或线段交集返回空向量。无界输入不在接口范围内。 $\text{Point}=\{x,y,d\}$ 表示 $(x/d,y/d)$, $d>0$, 分数不必约分; 按逆时针返回, 不重复首点, 相邻重复点已去除。排序与所有构造、判定使用整数和 `int128`, 不使用 `epsilon`。 $O(n \log n)$ 时间、 $O(n)$ 空间。浮点转换与面积计算需单独处理。

```
1 struct IntegerHalfplanes
2 {
3     using I = __int128_t;
4     using ll = long long;
5
6     struct Line
7     {
8         ll a, b, c;
9     };
10
11     struct Point
12     {
13         ll x, y, d;
14     };
15
16     static I cross(Line u, Line v) { return I(u.a) * v.b - I(u.b) * v.a; }
17
18     static int half(Line u) { return u.b < 0 || (u.b == 0 && u.a < 0); }
19
20     static Point meet(Line u, Line v)
21     {
22         I d = cross(u, v);
23         assert(d != 0);
24         I x = I(u.c) * v.b - I(u.b) * v.c;
25         I y = I(u.a) * v.c - I(u.c) * v.a;
26         if (d < 0)
27         {
28             d = -d;
29             x = -x;
30             y = -y;
31         }
32         return {(ll)x, (ll)y, (ll)d};
33     }
```

接上页同一结构体与函数：

```

35     static I value(Line u, Point p)
36     {
37         return I(u.a) * p.x + I(u.b) * p.y - I(u.c) * p.d;
38     }
39
40     static bool same(Point p, Point q)
41     {
42         return I(p.x) * q.d == I(q.x) * p.d && I(p.y) * q.d == I(q.y) * p.d;
43     }
44
45     static vector<Point> bounded_polygon(vector<Line> lines)
46     {
47         vector<Line> h;
48         for (auto u : lines)
49         {
50             assert(-10000000000 <= u.a && u.a <= 10000000000);
51             assert(-10000000000 <= u.b && u.b <= 10000000000);
52             assert(-10000000000 <= u.c && u.c <= 10000000000);
53             if (!u.a && !u.b)
54             {
55                 if (u.c < 0) return {};
56             }
57             else
58                 h.push_back(u);
59         }
60         sort(h.begin(),
61              h.end(),
62              [](Line u, Line v)
63              {
64                  if (half(u) != half(v)) return half(u) < half(v);
65                  I d = cross(u, v);
66                  if (d) return d > 0;
67                  ll x = u.a ? (u.a > 0 ? u.a : -u.a) : (u.b > 0 ? u.b : -u.b);
68                  ll y = v.a ? (v.a > 0 ? v.a : -v.a) : (v.b > 0 ? v.b : -v.b);
69                  return I(u.c) * y < I(v.c) * x;
70              });

```

接上页同一结构体与函数：

```

71     deque<Line> q;
72     for (int i = 0; i < (int)h.size(); i++)
73     {
74         Line u = h[i];
75         if (i && half(u) == half(h[i - 1]) && cross(u, h[i - 1]) == 0) continue;
76         while (q.size() >= 2 && value(u, meet(q[q.size() - 2], q.back())) > 0)
77             q.pop_back();
78         while (q.size() >= 2 && value(u, meet(q[0], q[1])) > 0) q.pop_front();
79         if (!q.empty() && cross(q.back(), u) == 0) return {};
80         q.push_back(u);
81     }
82     while (q.size() >= 3 && value(q.front(), meet(q[q.size() - 2], q.back())) > 0)
83         q.pop_back();
84     while (q.size() >= 3 && value(q.back(), meet(q[0], q[1])) > 0) q.pop_front();
85     if (q.size() < 3 || cross(q.front(), q.back()) == 0) return {};
86     vector<Point> answer;
87     for (int i = 0; i < (int)q.size(); i++)
88     {
89         Point p = meet(q[i], q[(i + 1) % q.size()]);
90         if (answer.empty() || !same(answer.back(), p)) answer.push_back(p);
91     }
92     if (answer.size() > 1 && same(answer.front(), answer.back())) answer.pop_back();
93     if (answer.size() < 3) return {};
94     return answer;
95 }
96 };

```

第七章 平面最近点对

7.1 整数最近点对（精确平方距离）

`closest_pair_i64(points)` 返回 `optional<__int128_t>` 的最短距离平方；少于两点返回 `nullopt`，重复点返回 0。点类型为 `IntegerPlane::Point`，坐标绝对值不超过 10^{12} 。按横坐标排序后递归分治，递归返回时按纵坐标排序； $O(n \log n)$ 时间、 $O(n)$ 辅助空间，全程精确整数。按值传入，不修改调用方序列。依赖 `IntegerPlane` 的比较与 `dist2`。

```
1 optional<IntegerPlane::I> closest_pair_i64(vector<IntegerPlane::Point> p)
2 {
3     using G = IntegerPlane;
4     using P = G::Point;
5     using I = G::I;
6     int n = p.size();
7     if (n < 2) return nullopt;
8     sort(p.begin(), p.end());
9     for (int i = 1; i < n; i++)
10         if (p[i] == p[i - 1]) return I(0);
11     vector<P> tmp(n);
12     auto by_y = [](P a, P b)
13     {
14         return tie(a.y, a.x) < tie(b.y, b.x);
15     };
16     function<I(int, int)> solve = [&](int l, int r) -> I
17     {
18         if (r - l <= 3)
19         {
20             I best = I(1) << 120;
21             for (int i = l; i < r; i++)
22                 for (int j = i + 1; j < r; j++) best = min(best, G::dist2(p[i], p[j]));
23             sort(p.begin() + l, p.begin() + r, by_y);
24             return best;
25         }
26         int m = (l + r) / 2;
27         long long x = p[m].x;
28         I best = min(solve(l, m), solve(m, r));
29         merge(p.begin() + l,
30              p.begin() + m,
31              p.begin() + m,
32              p.begin() + r,
33              tmp.begin() + l,
34              by_y);
35         copy(tmp.begin() + l, tmp.begin() + r, p.begin() + l);
36         vector<P> strip;
37         for (int i = l; i < r; i++)
38             if ((I(p[i].x) - x) * (I(p[i].x) - x) < best)
39             {
40                 for (int j = (int)strip.size() - 1; j >= 0; j--)
41                 {
42                     I dy = I(p[i].y) - strip[j].y;
```

```

43         if (dy * dy >= best) break;
44         best = min(best, G::dist2(p[i], strip[j]));
45     }
46     strip.push_back(p[i]);
47 }
48 return best;
49 };
50 return solve(0, n);
51 }

```

7.2 浮点平面最近点对

`solve(points)` 返回 `long double` 最短距离 (不是平方), 少于两个点返回 `nullopt`, 重复点返回零。Point={x,y}; 输入须有限, 坐标绝对值 $\leq 1e100$ 。递归分治, 按纵坐标归并, $O(n \log n)$ 时间、 $O(n)$ 额外空间。用 `hypotl` 求距离, 不先平方; 排序不使用 `epsilon`。浮点运算仍有舍入误差, 输入前已丢失的精度无法恢复。

```

1 struct ClosestPair
2 {
3     using R = long double;
4
5     struct Point
6     {
7         R x, y;
8     };
9
10    // Finite coordinates, |x|, |y| <= 1e100. Returns distance, not its square.
11    static optional<R> solve(vector<Point> p)
12    {
13        int n = p.size();
14        if (n < 2) return nullopt;
15        sort(p.begin(),
16             p.end(),
17             [](Point a, Point b) { return tie(a.x, a.y) < tie(b.x, b.y); });
18        for (int i = 1; i < n; i++)
19            if (p[i].x == p[i - 1].x && p[i].y == p[i - 1].y) return R(0);
20        auto by_y = [](Point a, Point b)
21        {
22            return tie(a.y, a.x) < tie(b.y, b.x);
23        };
24        auto distance = [](Point a, Point b)
25        {
26            return hypotl(a.x - b.x, a.y - b.y);
27        };
28        vector<Point> tmp(n);
29        function<R(int, int)> dfs = [&](int l, int r) -> R
30        {
31            if (r - l <= 3)
32            {
33                R best = numeric_limits<R>::infinity();
34                for (int i = l; i < r; i++)
35                    for (int j = l; j < i; j++) best = min(best, distance(p[i], p[j]));
36                sort(p.begin() + l, p.begin() + r, by_y);
37                return best;
38            }

```

分治、归并与跨分界线点对（接上页同一函数）：

```
39     int m = (l + r) / 2;
40     R x = p[m].x;
41     R best = min(dfs(l, m), dfs(m, r));
42     merge(p.begin() + l,
43           p.begin() + m,
44           p.begin() + m,
45           p.begin() + r,
46           tmp.begin() + l,
47           by_y);
48     copy(tmp.begin() + l, tmp.begin() + r, p.begin() + l);
49     vector<Point> strip;
50     for (int i = l; i < r; i++)
51         if (fabs1(p[i].x - x) <= best)
52         {
53             for (int j = (int)strip.size() - 1; j >= 0; j--)
54             {
55                 if (p[i].y - strip[j].y > best) break;
56                 best = min(best, distance(p[i], strip[j]));
57             }
58             strip.push_back(p[i]);
59         }
60     return best;
61 };
62 return dfs(0, n);
63 }
64 };
```

第八章 随机增量法

8.1 随机增量最小覆盖圆

`solve(points,rng,eps)` 返回 `optional<Circle>`, `Circle={o,r}`; 空集返回 `nullopt`, 单点或重复同一点返回半径零。`rng` 是调用者提供的 `mt19937_64` 随机数引擎。期望 $O(n)$ 时间, 最坏 $O(n^3)$, $O(n)$ 空间; 随机化不可省略。`eps` 默认 32 倍 long double 机器精度, 用于相对半径的包含判断。`diameter(a,b)` 求直径圆; `through(a,b,c)` 求外接圆, 共线时返回最远点对的直径圆。依赖浮点几何的 `Point/Circle` 与向量运算。输入及中间运算须有限且尺度适当; 接近共线时圆心可能不稳定, 无法恢复读入前的精度。

```
1 struct EnclosingCircle
2 {
3     using G = RealPlane;
4     using R = G::R;
5     using Point = G::Point;
6     using Circle = G::Circle;
7
8     static Circle diameter(Point a, Point b)
9     {
10         Point o = a + (b - a) / 2;
11         return {o, max(G::norm(o - a), G::norm(o - b))};
12     }
13
14     static Circle through(Point a, Point b, Point c)
15     {
16         Point u = b - a, v = c - a;
17         R scale = max({fabsl(u.x), fabsl(u.y), fabsl(v.x), fabsl(v.y)});
18         if (scale == 0) return {a, 0};
19         u = u / scale;
20         v = v / scale;
21         R det = 2 * G::cross(u, v);
22         if (det == 0)
23         {
24             Circle best = diameter(a, b);
25             for (Circle q : {diameter(a, c), diameter(b, c)})
26                 if (q.r > best.r) best = q;
27             return best;
28         }
29         R x = G::dot(u, u), y = G::dot(v, v);
30         Point offset{(x * v.y - y * u.y) / det, (u.x * y - v.x * x) / det};
31         Point o = a + offset * scale;
32         return {o, max({G::norm(o - a), G::norm(o - b), G::norm(o - c)})};
33     }
```

随机增量主过程（接上页同一结构体）：

```
35     static optional<Circle>
36     solve(vector<Point> p, mt19937_64 &rng, R eps = 32 * numeric_limits<R>::epsilon())
37     {
38         assert(eps >= 0 && eps < 1);
39         if (p.empty()) return nullopt;
40         shuffle(p.begin(), p.end(), rng);
41         Circle c{p[0], 0};
42         auto outside = [&](Point q)
43         {
44             return G::norm(q - c.o) > c.r * (1 + eps);
45         };
46         for (int i = 1; i < (int)p.size(); i++)
47             if (outside(p[i]))
48             {
49                 c = {p[i], 0};
50                 for (int j = 0; j < i; j++)
51                     if (outside(p[j]))
52                     {
53                         c = diameter(p[i], p[j]);
54                         for (int k = 0; k < j; k++)
55                             if (outside(p[k])) c = through(p[i], p[j], p[k]);
56                     }
57             }
58         return c;
59     }
60 };
```

第九章 配套知识与验证范围

9.1 数值算法与几何

排序必须使用严格弱序；不能以“差小于 eps 则相等”定义通用排序比较器。叉积与行列式容易发生抵消；整数题优先精确整数判定，浮点构造单独处理误差。对于简单格点多边形（无孔），Pick 定理 $A = I + B/2 - 1$ ，边界格点数 $B = \sum \gcd(|\Delta x|, |\Delta y|)$ ；不要重复计入边端点。自适应积分的局部误差估计需要函数足够规则；奇点、间断与强振荡必须先分段处理。三分搜索要求单峰性，整数域最后枚举剩余区间；二分答案要求判定单调性。

来源索引 本章为重新组织的公式速查，扩展范围依据 <https://oi-wiki.org/math/>。小模数 exLucas 与有限域拉格朗日插值已有双风格实现和本地验证；素数模平方根已有 Cipolla 实现；仍需补齐二次剩余的其他方法与定理、Min25、Newton 插值、子集卷积、行列式进阶、数值积分等内容及验证。本章公式不能代替这些待完成代码。

第十章 计算几何：精确判定与适用范围

10.1 整数系数的有界半平面交

实现见第 22 页。输入是不等式 $ax + by \leq c$ ，三个系数均为绝对值不超过 10^9 的整数。本接口要求交集有界，只输出正面积多边形；无解、点交集和线段交集都返回空向量。它不是一般的可行性分类接口，不可将其空返回直接解读为闭半平面交集必为空。无界输入不在约定范围内，也不偷偷添加一个人工边框。

两条非平行边界交点用齐次坐标 (X, Y, D) 表示：

$$D = a_1b_2 - b_1a_2, \quad X = c_1b_2 - b_1c_2, \quad Y = a_1c_2 - c_1a_2.$$

若 $D < 0$ ，三者同时取负，使 $D > 0$ 。测试 $aX + bY \leq cD$ 即可判断交点位于半平面内，无需除法。系数界保证 $|X|, |Y|, D \leq 2 \times 10^{18}$ ，可用 64 位保存；相等判断的交叉乘积至多为 4×10^{36} ，使用 128 位。因此本接口内的构造、排序和位置判定都是精确整数运算。

按法向量的半圆与叉积排序，比较器不使用 epsilon。同方向平行约束按正规化后的常数项保留更紧者，避免除以法向量长度。扫描时若新约束排除队首或队尾交点，就删除相应无效边界；每条边界最多入队、出队一次，排序后为线性时间。最后闭合队列，清除重复顶点；有界的非正面积结果不会输出多边形。零法向量表示常真约束或直接矛盾，先单独处理。

测试枚举所有边界对的交点，用任意精度整数逐条验证可行性，将可行顶点集合与返回集合比较，并检查循环顺序的严格凸性。两套版本通过 5000 组有界随机输入、平行和退化案例、近乎平行的十亿系数，以及十万条冗余约束的规模测试；普通与 ASan/UBSan 均通过。将有理数坐标转换为浮点、计算面积或处理浮点输入属于另一个数值问题，尚未由此验证。WIDA 的一般半平面交条目保留部分完成状态，浮点版本与无界分类仍需补齐。

10.2 最近点对、Minkowski 和与三维整数判定

整数最近点对见第 25 页；Minkowski 和见第 17 页。最近点对返回距离平方，不开方；少于两个点时返回空 optional，重复点返回零。分治使用递归，每层按纵坐标归并，总时间 $O(n \log n)$ ，额外空间 $O(n)$ 。坐标绝对值上限为 10^{12} ，作差后平方使用 128 位。

Minkowski 和输入必须是严格逆时针凸包，不重复首点，不含边上的冗余共线点；允许循环移动起点，也允许空集、单点和线段。输入坐标以及任意两个输入点的坐标和都须在 $[-10^{12}, 10^{12}]$ 内。非退化凸包按边方向归并，时间 $O(n + m)$ ；退化分支枚举点和后求凸包，时间 $O((n + m) \log(n + m))$ 。返回同样不重复首点的严格凸包。

两套代码通过任意精度整数距离枚举，以及对所有点和独立执行礼品包装法的对拍；覆盖退化、循环起点和大坐标。规模测试包含二十万点最近点对，以及 40001 顶点的抛物线凸包与自身求和，结果应为原凸包各坐标的两倍。这些属于本地验证，不表示新增在线 AC。

三维判定见第 13 页。点的三个坐标为 x, y, z ，绝对值均不超过 10^9 。`orient(a, b, c, d)` 返回有向六倍四面体体积，零表示四点共面，交换任意两个点会改变符号。`collinear` 判断三点共线，`on_segment` 包含端点，且支持零长度线段。传统版线段接口名为 `On_Segment`。

点差每个分量至多 2×10^9 ，叉积分量至多 8×10^{18} ，三重积绝对值至多 4.8×10^{28} ，在有符号 128 位范围内。公开的叉积、点积辅助函数不对任意 128 位向量保证不溢出；上述保证针对点差及其叉积。任意精度整数按 4×4 行列式的 24 个排列独立计算方向，并用二阶子式及坐标区间验证共线与在线段上。已通过立方体八个顶点的全部有序四元组、随机十亿级坐标、换序及退化测试，普通与 ASan/UBSan 均通过。这里不包含三维凸包构造。

10.3 浮点最近点对

实现见第 26 页。`ClosestPair::solve` 与传统版 `Closest_Pair::Solve` 接收 `Point{x,y}` 数组，返回最短距离，不是距离平方。少于两点返回空 optional；重复点返回零。输入为有限 `long double`，坐标绝对值不超过 10^{100} 。

递归前按横坐标排序，每次在递归改变数组前保存分界横坐标。左右区间分别递归后按纵坐标归并，再检查分界线附近的点对。时间 $O(n \log n)$ ，额外空间 $O(n)$ 。直接使用 `hypotl`，避免先平方造成上溢或下溢。排序比较器不使用 `epsilon`；不能把相近坐标随意视为相同点。

浮点结果仍有舍入误差，也无法恢复读入之前已经丢失的精度。整数坐标要求精确判定时，优先使用整数最近点对，最后按题目要求开方输出。一百位十进制浮点枚举所有点对，验证两套实现，覆盖小数、相邻可表示浮点数、极小非正规数、 10^{-100} 至 10^{100} 的尺度，以及二十万点规模；普通与 ASan/UBSan 均通过。本地完整 P1429 驱动测试包含 103 组输入和四位小数输出检查。在线评测的整数输入范围不能替代这里对小数与极端尺度的检查。

两套实现已在 P1429 通过全部 11 个测试点：[动态版记录 297602650](#)、[传统版记录 297603238](#)。单点最大用时均为 75 ms，内存峰值均为 12.32 MB。

10.4 公切线与过点切线

实现见第 11 页，依赖浮点几何的点、圆和向量运算。令两圆心差为 v ， $d = |v|$ ， $u = v/d$ 。外公切线取 $s = 1$ ，内公切线取 $s = -1$ ，单位法向量满足

$$n \cdot u = \frac{r_1 - sr_2}{d} = c, \quad n = cu \pm \sqrt{(1-c)(1+c)} u^\perp.$$

切点为 $p = o_1 + r_1 n$ 、 $q = o_2 + sr_2 n$ 。直线以 $n \cdot (x - p) = 0$ 表示，避免在两圆相切、两个切点相同时丢失方向。结果中的 **side** 为 1 或 -1，表示外公切线或内公切线。每个非相切分支有两条线，相切分支只输出一次。

半径零解释为点。有一个点圆时，只枚举外公切线分支，避免内外分支重复；两个不同点直接返回连线，**side**=0。严格同心且等半径返回 **infinite=true**，包括重合的两个点圆；严格同心但半径不同没有公切线。**from_point(p,c)** 把第一圆设为点圆，圆上的切点读返回的 **b**。点在圆上时返回一条，点在圆内时空；点与零半径圆重合时返回无限条过点直线。

输入须有限，坐标和半径绝对值不超过 10^{100} ，半径非负。容差 $0 < \varepsilon < 1$ 默认 10^{-12} ：当 $|r_1 - sr_2|$ 与 d 的相对差在容差内时，合并为相切。因此极接近相切的输入可能返回一条近似切线，不承诺恢复舍入前的几何分类；使用方程时宜计算 $n \cdot (x - p)$ ，避免先计算大常数项造成额外消减。先比较半径差和圆心距，再做除法，避免极小圆心距造成比值溢出。

计数对拍使用任意精度整数比较圆心距平方与半径和、差的平方；一百位十进制运算验证切点到圆心的距离、圆心到切线的距离、法向量和切线一致性。两套实现通过 3000 组随机圆，在三个尺度下交换顺序测试，并覆盖重合、内外相切、点圆、极近同心与容差边界。普通与 ASan/UBSan 均通过；尚无此接口的在线 AC 记录。

10.5 随机增量最小覆盖圆

实现见第 28 页。随机打乱点集后依次加入。新点在当前圆外时，它必须成为新覆盖圆的边界点；以该点为固定边界重扫前缀，再遇到圆外点就固定两个边界点，最后由第三个圆外点确定外接圆。随机化使期望时间为 $O(n)$ ，最坏时间仍为 $O(n^3)$ 。传入初始化好的 `mt19937_64`，不可省略打乱。本实现使用三层循环，没有递归栈需求。

空集返回空 optional，单点及重复同一点的半径为零。直径圆用两端点到计算圆心的较大距离作为半径。三点外接圆先把第一点平移到原点，再按最大坐标差缩放后解两条垂直平分线方程；共线时使用最远点对的直径圆。`through` 对非共线三点求外接圆，不是任意三点的最小覆盖圆；例如钝角三角形的最小覆盖圆可能仅由最长边支撑，须调用主过程。

包含判断使用相对半径容差，默认 32 倍机器精度。输入与中间运算须有限、尺度适当；近共线三点的外接圆仍可能数值敏感，浮点比较不能恢复读入之前的精度。[P1742](#) 的驱动已准备，输出半径与圆心，该题坐标绝对值不超过 10^4 ，最多两位小数。

本地参考答案枚举所有单点、点对直径圆和三点外接圆，用一百位十进制运算筛出覆盖全部点且半径最小者，检查圆心、半径及覆盖性。两套版本通过 700 组随机小数点集，每组使用 12 个随机种子，另测共线、近共线、钝角/锐角三角形、平移后的小三角形、十万点直线与重复点。普通与 ASan/UBSan 均通过。两套代码已在 [P1742](#) 通过全部 10 个测试点：[动态版记录 297608403](#)、[传统版记录 297608801](#)。单点最大用时分别为 52 ms、49 ms，内存峰值为 6.50 MB、6.49 MB。该评测验证半径与圆心，不覆盖一并展开但未调用的其他几何接口。随机增量算法可参考 [Project Nayuki 的算法说明](#)。

10.6 圆与简单多边形相交面积

实现见第 9 页。先把圆心移到原点，再沿多边形有向边累加贡献，最后取绝对值。边与圆的交点把边分成至多三段。若小段中点在圆内，贡献是 $\text{cross}(p, q)/2$ ；否则贡献是有向扇形面积

$$\frac{r^2}{2} \text{atan2}(\text{cross}(p, q), p \cdot q).$$

必须保留符号，不能先对每条边取绝对值，否则凹多边形和圆外的边无法抵消。使用补偿求和减小大量边累加时的误差，总时间 $O(n)$ 、额外空间 $O(1)$ 。

求交点时令边的单位方向为 u ，有向距离 $h = \text{cross}(u, a)$ ，垂足直接构造为 $hu^\perp$ ，交点为

$$hu^\perp \pm u\sqrt{(r - |h|)(r + |h|)}.$$

这避免用远端点加很长的位移来重建圆附近的交点。沿边的投影参数用于筛选位于线段内部的交点；端点本身已在分段列表内。相切交点重复不改变面积，零长度边返回零。

多边形须简单，支持凹形、两种方向和相邻重复顶点；半径非负，零半径或少于三个顶点时面积为零。输入与中间运算须有限且尺度适当，特别是平移后的乘积和面积要可表示。极小相交面积可能受到扇形贡献相消和近相切计算的影响，不作相对误差的统一保证。

测试包括完整圆、半圆、四分之一圆、内含多边形、相离、相切和高精度圆缺面积。独立参考实现沿竖直方向求多边形截线区间，再与圆的截线相交并数值积分，检查 150 组含凹形的随机多边形，另检查方向反转、重复点和平移。规模与数值测试含十万条边、极长的过圆心边和不同尺度。两套实现普通与 ASan/UBSan 均通过；尚无此模块的在线 AC。对应 kuangbin 二维几何中的圆与三角形、多边形相交面积部分。

算法索引

circle__intersections, 5
circle__overlap__area, 6
CirclePolygon, 9
CircleTangents, 11
closest__pair__i64, 25
ClosestPair, 26
convex__contains__i64, 16
convex__diameter2, 21

EnclosingCircle, 28

integer__hull, 15
IntegerGeometry3D, 13
IntegerHalfplanes, 22
IntegerPlane, 6

line__circle__intersections, 4
line__intersection__real, 3
line__projection, 2

minkowski__sum, 17

numerical stability, 30

Pick theorem, 30
polygon__area2, 7
polygon__contains, 8

RealPlane, 1

segment__distance__real, 14
SupportHull, 19